



ZPI-Desktop-App

Dokumentacja Techniczna — Aplikacja Desktopowa

Angular 20

Ionic 8

TypeScript

Socket.IO

REST API

JWT

Docker

WebCrypto

Projekt Inżynierski — ZPI | Rok akademicki 2025/2026 | Wersja 1.0



Spis treści

1. Słownik pojęć i technologii
2. Stack technologiczny
3. Architektura aplikacji
4. Warstwa rdzenia (core)
5. Strony wejściowe — Login i Zmiana hasła
6. Moduły funkcjonalne (features)
7. Routing i ochrona tras
8. Bezpieczeństwo
9. Wdrożenie (Docker + nginx)

1. Słownik pojęć i technologii

Ta sekcja wyjaśnia wszystkie kluczowe terminy techniczne użyte w projekcie, tak aby dokumentacja była zrozumiała dla każdego czytelnika.

Podstawowe pojęcia webowe

Frontend vs. Backend

Frontend

To część aplikacji, którą widzi i z którą wchodzi w interakcję użytkownik — to co działa w przeglądarce internetowej lub oknie desktopowym. W tym projekcie frontend to kod Angular/Ionic działający w przeglądarce. Frontend wysyła żądania do backendu i wyświetla otrzymane dane.

Backend

To serwer — niewidoczna część aplikacji działająca gdzieś na maszynie serwerowej. Przetwarza logikę biznesową, zarządza bazą danych i udostępnia dane frontendowi

przez API. W tym projekcie backend to serwer Python (FastAPI) dostępny pod adresem `apiBaseUrl`.

CRUD

CRUD — Create, Read, Update, Delete

Cztery podstawowe operacje na danych, które realizuje niemal każda aplikacja:

- **Create (Utwórz)** — dodanie nowego rekordu (np. dodanie nowej sali wykładowej)
- **Read (Odczyt)** — pobranie i wyświetlenie danych (np. lista przedmiotów)
- **Update (Aktualizacja)** — edycja istniejącego rekordu (np. zmiana liczby miejsc w sali)
- **Delete (Usunięcie)** — usunięcie rekordu (np. usunięcie użytkownika)

W projekcie CRUD wykonuje się przez wysyłanie żądań HTTP do API.

API i REST API

API (Application Programming Interface)

Zestaw reguł definiujący, jak dwa programy mogą się ze sobą komunikować. To swoisty "kontrakt" między frontendem a backendem — frontend wie, pod jaki adres wysłać żądanie i jaki format danych otrzyma w odpowiedzi.

REST API (REpresentational State Transfer)

Popularny styl architektury API oparty na protokole HTTP. Główne zasady:

- Zasoby identyfikowane przez adresy URL, np. `/subjects/42`
- Operacje realizowane przez metody HTTP: **GET** (pobierz), **POST** (utwórz), **PUT/PATCH** (aktualizuj), **DELETE** (usuń)
- Komunikacja bezstanowa — każde żądanie jest niezależne
- Dane wymieniane w formacie **JSON**

W projekcie wszystkie serwisy `*-api.service.ts` wywołują REST API backendu.

JSON

JSON (JavaScript Object Notation)

Lekki format wymiany danych — tekstowy zapis obiektów. Przykład odpowiedzi API zwracającej salę:

```
{
  "id": 5,
  "room_number": "A105",
  "seats_count": 30,
  "room_type": "Laboratorium"
}
```

Przeglądarka zamienia ten tekst na obiekt JavaScript, z którym pracuje Angular.

HTTP i metody HTTP

Metoda	Znaczenie	Przykład w projekcie
GET	Pobiera dane (nie modyfikuje)	Pobranie listy przedmiotów
POST	Tworzy nowy zasób / wysyła dane	Logowanie, dodanie sali
PUT	Całkowita aktualizacja zasobu	Aktualizacja klucza publicznego
PATCH	Częściowa aktualizacja zasobu	Zmiana liczby godzin w przydziale
DELETE	Usuwa zasób	Usunięcie notatki niedostępności

Uwierzytelnianie i bezpieczeństwo

JWT — JSON Web Token

JWT (JSON Web Token)

Kompaktowy, kryptograficznie podpisany token (ciąg znaków) służący do potwierdzania tożsamości użytkownika. Składa się z trzech części oddzielonych kropką:

`header.payload.signature`

- **Header** — informacja o algorytmie podpisu (np. RS256)
- **Payload** — dane użytkownika: `user_id`, `role`, `exp` (czas wygaśnięcia)
- **Signature** — podpis weryfikujący autentyczność — tylko serwer może go wygenerować

Frontend przechowuje token w pamięci RAM i wysyła go w każdym żądaniu do API. Po wygaśnięciu (np. 15 minut) zostaje automatycznie odświeżony przez Refresh Token.

Access Token vs. Refresh Token

Token	Czas życia	Gdzie przechowywany	Cel
Access Token	Krótki (np. 15 min)	Pamięć RAM aplikacji (nie trwa)	Autoryzacja każdego żądania API
Refresh Token	Długi (np. 7 dni)	HttpOnly Cookie (nie dostępny dla JS)	Uzyskanie nowego Access Tokena bez re-logowania

HttpOnly Cookie — dlaczego bezpieczniejszy?

Cookie z flagą `HttpOnly` nie jest dostępne przez JavaScript — przez co ewentualny złośliwy skrypt na stronie (atak XSS) nie może go ukraść. Przeglądarka wysyła go automatycznie przy każdym żądaniu do odpowiedniej domeny.

Interceptor HTTP

Interceptor

Mechanizm przechwytyjący każde żądanie HTTP "w locie" przed jego wysłaniem lub po odebraniu odpowiedzi. To odpowiednik "bramki" przez którą przechodzi każde żądanie. W projekcie `auth.interceptor.ts` automatycznie:

- Dodaje `withCredentials: true` (wysyła cookie) do każdego żądania API

- Przy błędzie 401 (brak autoryzacji) próbuje automatycznie odświeżyć token
- Jeśli refresh się nie powiedzie, przekierowuje do strony logowania

RSA-OAEP — Szyfrowanie asymetryczne

Szyfrowanie asymetryczne RSA

System szyfrowania używający **pary kluczy**: klucza publicznego i prywatnego.

- **Klucz publiczny** — może być udostępniony wszystkim; służy do *szyfrowania*
- **Klucz prywatny** — tajny, tylko właściciel; służy do *odszyfrowania*
- To co zaszyfrowano kluczem publicznym **A**, odszyfrowuje tylko klucz prywatny **A**

W projekcie: każdy użytkownik przy logowaniu generuje własną parę RSA-OAEP 2048-bit. Wiadomości czatu są szyfrowane kluczem publicznym odbiorcy — przeczytać je może tylko odbiorca swoim prywatnym kluczem. To jest **end-to-end encryption (E2EE)**.

IndexedDB

IndexedDB

Wbudowana baza danych przeglądarki pozwalająca trwale przechowywać dane po stronie klienta. W projekcie: klucz prywatny RSA jest zapisywany w IndexedDB (baza `zpi-crypto`) — trwa między sesjami ale nie opuszcza urządzenia użytkownika.

Real-time i komunikacja

WebSocket i Socket.IO

WebSocket

Protokół umożliwiający dwukierunkową, trwałą komunikację między przeglądarką a serwerem w czasie rzeczywistym. W przeciwieństwie do HTTP (gdzie klient zawsze inicjuje żądanie), WebSocket pozwala serwerowi samodzielnie wysyłać dane do klienta.

Przykład: gdy ktoś wyśle wiadomość na czacie, serwer natychmiast "pcha" ją do przeglądarki odbiorcy — bez potrzeby ciągłego odpytywania serwera co sekundę ("polling").

Socket.IO

Biblioteka zbudowana nad WebSocket, dodająca automatyczne ponowne łączenie, obsługę pokoi (rooms), potwierdzenia dostarczenia i mechanizmy fallback. W projekcie używana w `websocket.service.ts` do obsługi czatu real-time.

RxJS i Observable

RxJS / Observable / BehaviorSubject

Biblioteka do programowania reaktywnego. Zamiast jednorazowych wartości, operuje się na "strumieniach" danych (Observable) — sekwencjach wartości emitowanych w czasie.

- **Observable** — strumień danych, można się "subskrybować" i reagować na każdą nową wartość
- **BehaviorSubject** — Observable przechowujący zawsze aktualną wartość (np. stan zalogowania)
- **Subject** — "kanał" do emitowania zdarzeń (np. nowa wiadomość na czacie)
- **pipe/map/catchError** — operatory do transformacji i obsługi błędów w strumieniach

Przykład w projekcie: `isAuthenticated$` to BehaviorSubject — menu automatycznie pojawia się gdy jego wartość zmieni się na `true`.

Frameworki i biblioteki UI

Angular

Angular (framework frontendowy Google)

Pełnoprawny framework JavaScript/TypeScript do budowania aplikacji SPA. Główne koncepcje:

- **Komponent** — samodzielna jednostka UI (klasa TS + szablon HTML + style CSS)
- **Serwis (Service)** — klasa z logiką biznesową, wstrzykiwana przez Dependency Injection
- **Dependency Injection (DI)** — mechanizm automatycznego dostarczania serwisów do komponentów
- **Router** — zarządza nawigacją między widokami bez przeładowania strony
- **Standalone Components** — nowszy wzorec (Angular 14+), bez konieczności tworzenia modułów NgModule

Ionic Framework

Ionic (biblioteka komponentów UI)

Biblioteka gotowych komponentów interfejsu użytkownika zaprojektowanych dla aplikacji mobilnych i desktopowych. Dostarcza komponenty takie jak: `ion-menu`, `ion-list`, `ion-button`, `ion-modal` itd. Automatycznie dostosowuje wygląd do platformy (iOS/Android/Web). W projekcie używany jako warstwa UI Angulara.

SPA — Single Page Application

SPA (Single Page Application)

Aplikacja webowa, która ładuje się raz jako pojedynczy plik HTML, a następnie dynamicznie podmienia treść bez przeładowania całej strony. Nawigacja jest obsługiwana przez JavaScript (Angular Router). Efekt: aplikacja działa jak program desktopowy — natychmiastowe przejścia, brak "mignięć" strony. Plik `nginx.conf` jest skonfigurowany specjalnie dla SPA — każda ścieżka URL przekierowuje do `index.html`.

TypeScript

TypeScript

Nadzbior języka JavaScript dodający **statyczne typowanie**. Kompiluje się do czystego JavaScript. Zalety w projekcie:

- Interfejsy (`interface RoomDto`) opisują strukturę danych — błędy typów wykrywane w edytorze
- Autouzupełnianie kodu
- Łatwiejsze refaktoryzacje

Infrastruktura i wdrożenie

Docker i konteneryzacja

Docker

Narzędzie do "konteneryzacji" — pakowania aplikacji razem ze wszystkimi zależnościami w przenośny kontener. Kontener działa tak samo na każdej maszynie niezależnie od systemu operacyjnego. `Dockerfile` zawiera instrukcje budowania obrazu: zainstaluj Node, zbuduj aplikację Angular, skonfiguruj nginx.

nginx

nginx (serwer HTTP)

Wydajny serwer webowy serwujący statyczne pliki. Po zbudowaniu aplikacja Angular to zestaw plików HTML/JS/CSS — nginx je serwuje przeglądarce. Skonfigurowany w projekcie specjalnie dla SPA: każde żądanie do nieznanej ścieżki zwraca `index.html`, co pozwala Angularnemu routerowi obsłużyć nawigację.

Capacitor

Capacitor (Ionic)

Warstwa pozwalająca uruchomić aplikację webową (Angular/Ionic) jako natywną aplikację mobilną (iOS/Android) lub desktopową, z dostępem do natywnych API urządzenia (aparat, wibracje, pasek statusu). W tym projekcie zainstalowany jako opcja na przyszłość — projekt działa głównie jako web app.

2. Stack technologiczny

Frontend Framework

Angular 20

Główny framework do budowania UI. Standalone Components, reaktywne formularze, lazy loading tras.

UI Components

Ionic Framework 8

Biblioteka komponentów — menu, listy, buttony, modale, ikony ionicons. Responsive design.

Język programowania

TypeScript

Statycznie typowany JavaScript. Interfejsy DTO dla danych API, silne typowanie serwisów.

Real-time

Socket.IO Client 4

Dwukierunkowa komunikacja z serwerem — czat w czasie rzeczywistym, powiadomienia push.

Programowanie reaktywne

RxJS 7.8

Obsługa asynchronicznych strumieni danych, operatory pipe, BehaviorSubject.

Kryptografia

Web Crypto API

Wbudowane API przeglądarki — generowanie kluczy RSA-OAEP 2048-bit, szyfrowanie E2EE.

Konteneryzacja

Docker + nginx

Wieloetapowy build: Node.js buduje app, nginx serwuje pliki statyczne w produkcji.

Mobile bridge

Capacitor (Ionic)

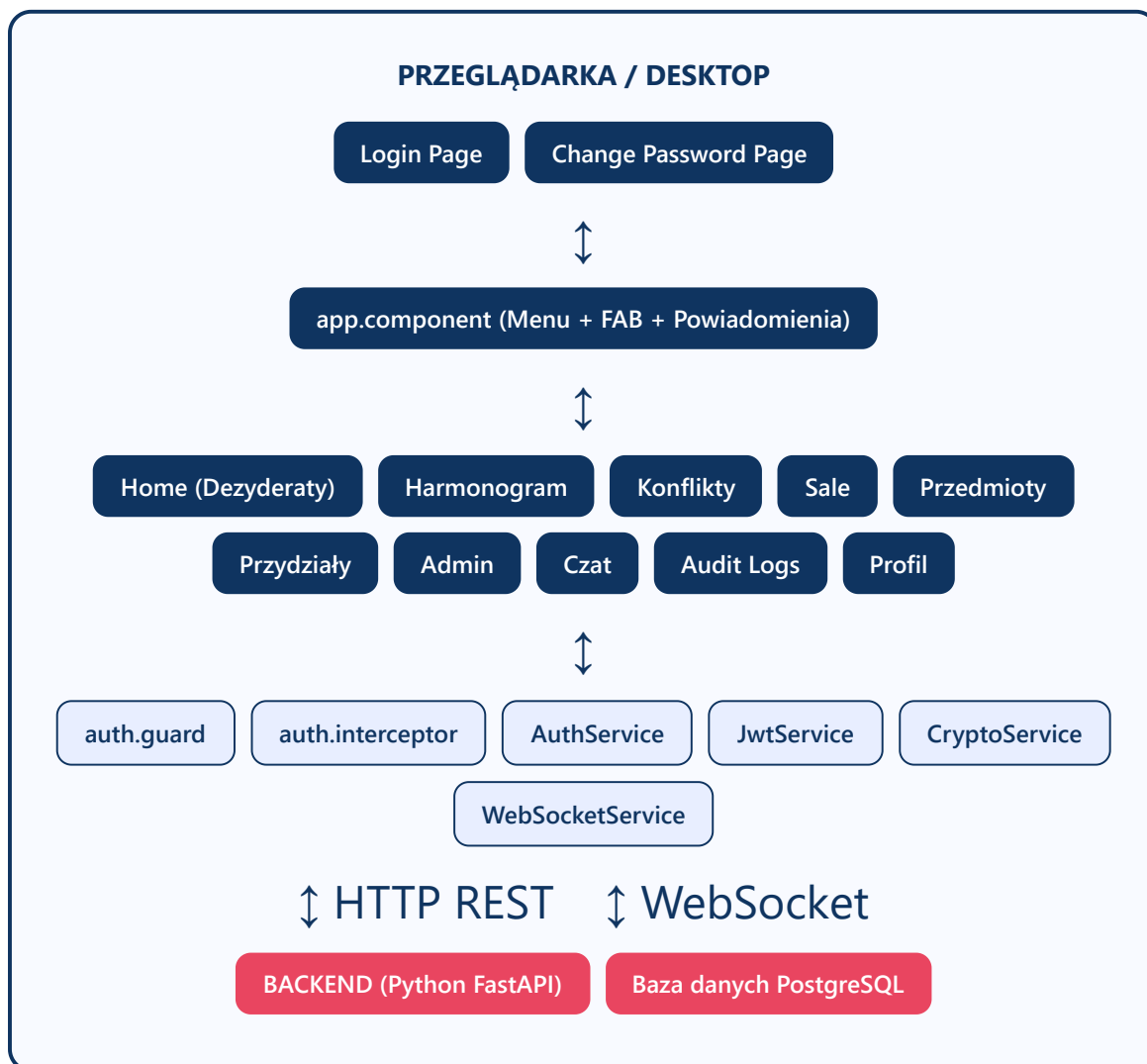
Opcjonalne wdrożenie jako aplikacja mobilna iOS/Android z dostępem do natywnych API.

Zależności produkcyjne (package.json)

Pakiet	Wersja	Rola
@angular/core	^20.0.0	Core frameworku Angular
@angular/router	^20.0.0	Routing SPA
@angular/forms	^20.0.0	Formularze reaktywne i template-driven
@ionic/angular	^8.0.0	Komponenty UI Ionic
ionicons	^7.0.0	Biblioteka ikon SVG
socket.io-client	^4.8.3	Klient WebSocket (Socket.IO)
rxjs	~7.8.0	Programowanie reaktywne
zone.js	~0.15.0	Detekcja zmian w Angularze
@capacitor/core	latest	Bridge do natywnych funkcji urządzenia

3. Architektura aplikacji

Diagram warstw



Struktura katalogów

```
src/app/
├── core/
│   ├── auth/
│   ├── interceptors/
│   ├── services/
│   └── validators/
├── features/
└── home/
```

- ← Logika współdzielona (singleton services)
- ← auth.guard, auth.service (JWT), jwt.service
- ← auth.interceptor (auto-refresh tokenów)
- ← Serwisy API dla każdego obszaru backendu
- ← Walidatory formularzy (hasło, email, imię)
- ← Moduły funkcjonalne (lazy-loaded)
- ← Dezyderaty (dostępność wykładowcy)

		hermonogram/	← Harmonogram tygodniowy z Rapłą
		konflikty/	← Widok konfliktów w planie
		sale/	← CRUD sal wykładowych
		subjects/	← CRUD przedmiotów
		teaching-loads/	← Przydziały godzin
		admin/	← Panel admina (użytkownicy, notatki)
		chat/	← Czat real-time z E2EE
		audit/	← Historia zmian
		profile/	← Profil i ustawienia
		pages/	← Strony bez wymogu auth
		login/	← Formularz logowania
		change-password/	← Wymuszona zmiana hasła
		app.routes.ts	← Definicja wszystkich ścieżek URL
		app.component.*	← Root: menu boczne, powiadomienia, FAB

Wzorzec Standalone Components

Projekt używa **Standalone Components** — nowoczesnego wzorca Angular 14+, gdzie każdy komponent deklaruje własne zależności (importy) bez potrzeby tworzenia modułów `NgModule`. Dzięki temu możliwy jest **lazy loading** — ładowanie kodu strony dopiero gdy użytkownik do niej nawiguje, co przyspiesza start aplikacji.

4. Warstwa rdzenia (core/)

4.1 Uwierzytelnianie — auth/

auth.service.ts (core/auth)

Serwis backendu JWT. Zarządza cyklem życia tokenów.

Metoda	Endpoint	Opis
<code>login(payload)</code>	POST <code>/auth/</code>	Wysyła login+hasło, otrzymuje access token
<code>refreshAccessToken()</code>	POST <code>/auth/refresh</code>	Odświeżenie tokena z refresh cookie
<code>fetchCurrentUser()</code>	GET <code>/auth/me</code>	Pobiera dane zalogowanego użytkownika
<code>logout()</code>	POST <code>/auth/logout</code>	Wyczyszczenie sesji po stronie serwera
<code>isAuthenticated()</code>	—	Sprawdza czy token nie wygał (z buforem 10s)

jwt.service.ts

Dekoduje tokeny JWT bez zewnętrznych bibliotek. Używa wbudowanej funkcji `atob()` przeglądarki do dekodowania Base64 payloadu. Metody: `decodePayload`, `getExpirationDate`, `isExpired`, `getUserId`, `getRole`.

auth.guard.ts

Route Guard — ochrona tras

Sprawdzany zanim Angular wyrenderuje jakikolwiek chroniony widok. Logika:

1. Jeśli token ważny + `mustChangePassword` → przekieruj na `/change-password`
2. Jeśli token ważny → przepuść
3. Jeśli token wygał → spróbuj odświeżyć z refresh cookie
4. Jeśli refresh się nie powiódł → przekieruj na `/login`

4.2 HTTP Interceptor — interceptors/

auth.interceptor.ts

Przechwytuje **każde** żądanie HTTP do `apiBaseUrl`. Automatycznie obsługuje 401 z deduplicją refreshu — wiele jednoczesnych żądań 401 wywołuje tylko **jeden** refresh request (dzięki `shareReplay(1)` i zmiennej `refreshInFlight$`).

4.3 Serwisy API — services/

Serwis	Endpoint backendu	Operacje CRUD
<code>auth.service.ts</code>	<code>/auth/me</code> , <code>/users/:id/public-key</code>	Sesja, motyw, avatar, klucz RSA
<code>subjects-api.service.ts</code>	<code>/subjects</code> , <code>/activities</code>	CRUD przedmiotów i słowników
<code>rooms-api.service.ts</code>	<code>/rooms</code>	CRUD sal, typów, sprzętu, wydziałów
<code>teaching-loads-api.service.ts</code>	<code>/teaching-loads</code>	CRUD przydziałów godzin dydaktycznych
<code>dezyderata.service.ts</code>	<code>/dezyderaty</code> , <code>/dezyderaty/semestr</code>	CRUD dezyderatów i semestrów
<code>unavailability-notes-api.service.ts</code>	<code>/unavailability-notes</code>	CRUD notatek niedostępności
<code>notifications-api.service.ts</code>	<code>/notifications</code>	Read + oznacz jako przeczytane
<code>audit-api.service.ts</code>	<code>/audit-logs</code>	Read + potwierdzenie obejrzenia
<code>rapla-api.service.ts</code>	<code>/rapla</code>	Import pliku Rapla, pobieranie rezerwacji
<code>users-admin-api.service.ts</code>	<code>/users</code> (admin)	CRUD kont użytkowników (admin)
<code>projects-api.service.ts</code>	<code>/api/projects</code>	Read lista projektów
<code>websocket.service.ts</code>	Socket.IO WS	Real-time: wiadomości, typing, presence
<code>crypto.service.ts</code>	IndexedDB + <code>/users/:id/public-key</code>	Generowanie i zarządzanie kluczami RSA

4.4 Walidatory formularzy — validators/

Validator	Reguła
<code>nameValidator()</code>	Polskie znaki, myślnik, spacja, 2–100 znaków
<code>strictEmailValidator()</code>	Format email wg regex
<code>passwordStrengthValidator()</code>	Min. 8 znaków, wielka litera, cyfra, znak specjalny
<code>passwordMatchValidator()</code>	Zgodność dwóch pól hasła (poziom FormGroup)
<code>passwordNotContainsNameValidator()</code>	Hasło nie może zawierać imienia/nazwiska
<code>calcPasswordStrength()</code>	Zwraca 0–4 (wskaźnik siły hasła)

5. Strony wejściowe — Login i Zmiana hasła

5.1 login.page — ścieżka `/login`

Formularz logowania. Pola: **login** i **hasło** (z przełącznikiem widoczności). Po poprawnym logowaniu: jeśli flaga `mustChangePassword` jest ustawiona, użytkownik trafia na `/change-password`, w przeciwnym razie na `/home`. Błędne dane → komunikat w języku polskim. Spinner podczas oczekiwania na odpowiedź API.

Strona dostępna bez uwierzytelnienia — nie jest chroniona przez `authGuard`.

5.2 change-password.page — ścieżka `/change-password`

Wymuszona zmiana hasła przy pierwszym logowaniu (lub gdy administrator zresetuje konto). Używa **reaktywnego formularza** (`ReactiveFormsModule`):

- Pole **Nowe hasło** — walidowane przez `passwordStrengthValidator` i `passwordNotContainsNameValidator`
- Pole **Potwierdź hasło** — sprawdzane przez `passwordMatchValidator`
- **Wskaźnik siły hasła** — kolorowy pasek (słabe/średnie/mocne/bardzo mocne)

Po pomyślnej zmianie: flaga `mustChangePassword` jest zerowana, użytkownik trafia na `/home`.

6. Moduły funkcjonalne (features/)

6.1 home/ — Dezyderaty

Ścieżka URL	<code>/home</code>
Dostęp	Wszyscy zalogowani użytkownicy
Cel	Wykładowca określa swoją dostępność godzinową per semestr i dzień tygodnia
API	<code>DezyderataService</code> → <code>/dezyderaty</code> , <code>/dezyderaty/semestry</code>

Dezyderata to preferencja dostępności: użytkownik określa dla danego semestru, w jakich dniach tygodnia i godzinach jest dostępny lub niedostępny. Planiści uwzględniają te dane przy układaniu harmonogramu.

6.2 hermonogram/ — Harmonogram zajęć

Ścieżka URL	<code>/hermonogram</code>
Dostęp	Admin, planer, rapla_editor (nie wykładowca)
Cel	Widok tygodniowego kalendarza z zaplanowanymi zajęciami z systemu Rapla
API	<code>RaplaApiService</code> → <code>/rapla/reservations</code> , <code>/rapla/file/import</code>

Funkcje strony:

- **Siatka godzinowa** — godziny 7:00–21:00, kolumny = dni tygodnia
- **Mini-kalendarz** — nawigacja po tygodniach, zaznaczenie aktualnego dnia
- **Import Rapla** — wgranie pliku eksportu z systemu Rapla (uczelniany system planowania); rezerwacje renderowane na siatce z obliczoną pozycją i wysokością (offset w minutach)
- **Kolizje** — nakładające się zdarzenia renderowane obok siebie z obliczoną szerokością

6.3 konflikty/ — Konflikty harmonogramu

Ścieżka
URL

/konflikty

Dostęp Admin, planer (nie wykładowca)

Cel Przeglądanie wykrytych konfliktów w harmonogramie (dwie rezerwacje w tym samym czasie/miejsce)

6.4 sale/ — Sale wykładowe (CRUD)

Dostęp Tylko admin

API RoomsApiService → /rooms

Strona	Ścieżka	Funkcja
sale-list.page	/sale	Lista wszystkich sal z wyszukiwarką, edycja inline, usuwanie
sale.page	/sale/new , /sale/:id/edit	Formularz tworzenia/edycji: numer sali, liczba miejsc, typ, sprzęt specjalny, wydziały, aktywności
sale-dictionaries.page	/sale/dictionaries	Zarządzanie słownikami: typy sal, sprzęt specjalny, typy aktywności

6.5 subjects/ — Przedmioty (CRUD)

Dostęp Tylko admin

API SubjectsApiService → /subjects , /activities

Strona / Komponent	Ścieżka	Funkcja
<code>subject-list.page</code>	<code>/subjects</code>	Lista przedmiotów z filtrowaniem i sortowaniem
<code>subject.page</code>	<code>/subjects/new</code> , <code>/subjects/:id/edit</code>	Formularz: nazwa, typ aktywności, właściwości wymaganej sali, blokada, cykliczność
<code>subject-dictionaries.page</code>	<code>/subjects/dictionaries</code>	Słowniki przedmiotów
<code>unavailability-note-modal</code>	Modal	Dodanie notatki o niedostępności z poziomu widoku przedmiotu

6.6 teaching-loads/ — Przydział godzin dydaktycznych

Ścieżka URL `/teaching-loads`

Dostęp Tylko admin

API `TeachingLoadsApiService` , `SubjectsApiService` , `DezyderataService` ,
`AuditApiService`

Funkcjonalności strony:

- **Tabela przydziałów** — kolumny: wykładowca, przedmiot, typ aktywności, semestr, liczba godzin
- **Edycja inline** — kliknięcie wiersza uruchamia tryb edycji bezpośrednio w tabeli (bez otwierania formularza)
- **Filtry wielopoziomowe** — po nauczycielu, przedmiocie, aktywności, semestrze, zakresie godzin
- **Historia zmian per wpis** — przycisk rozwijający audit log dla konkretnego przydziału
- **Tworzenie nowych wierszy** — "nowy wiersz" bezpośrednio w tabeli

6.7 admin/ — Panel administratora

Dodawanie użytkowników — `/admin/users/new`

Formularz tworzenia nowego konta użytkownika. Pola: imię, nazwisko, email, hasło, **role** (multi-select), **wydziały** (multi-select), **tytuły naukowe**. Używa serwisu

`UsersAdminApiService` → endpoint `/users` (POST).

Notatki niedostępności — `/admin/unavailability-notes`

Widok listy wszystkich notatek niedostępności zgłoszonych przez wykładowców.

Administrator może zmieniać status notatki:

Status	Znaczenie
<code>pending</code>	Oczekuje na decyzję admina
<code>accepted</code>	Zaakceptowana — wykładowca wolny w tym terminie
<code>rejected</code>	Odrzucona — wykładowca musi być dostępny
<code>acknowledged</code>	Przyjęte do wiadomości (notatka wymuszona)

Dwa typy notatek: `request` (prośba wykładowcy) i `forced` (narzucona przez admina).

6.8 chat/ — Czat real-time z szyfrowaniem E2EE

Ścieżka URL `/chat`

Dostęp Wszyscy zalogowani

Serwisy `ChatApiService`, `WebSocketService`, `CryptoService`

Architektura komponentów czatu:

Komponent	Rola
<code>chat.page</code>	Strona-wrapper — nagłówek Ionic, osadza ChatComponent
<code>chat.component</code>	Główna logika — zarządzanie konwersacjami przez WebSocket
<code>chat-rooms-view</code>	Lista pokoi/konwersacji z podglądem ostatniej wiadomości
<code>chat-contacts-view</code>	Lista kontaktów użytkownika
<code>chat-create-room-view</code>	Formularz tworzenia nowego pokoju czatu
<code>chat-panel-tabs</code>	Zakładki Pokoje / Kontakty / Nowy pokój
<code>chat-panel-header</code>	Nagłówek panelu z nazwą rozmowy

Jak działa szyfrowanie E2EE w czacie:

Proces szyfrowania wiadomości

1. Przy logowaniu `CryptoService` sprawdza, czy w **IndexedDB** istnieje klucz prywatny RSA
2. Jeśli nie — generuje nową parę RSA-OAEP 2048-bit przez **Web Crypto API**
3. Klucz prywatny zapisywany w IndexedDB (pozostaje na urządzeniu, nigdy nie wychodzi)
4. Klucz publiczny uploadowany na backend (`PUT /users/:id/public-key`)
5. Przed wysłaniem wiadomości: pobranie klucza publicznego odbiorcy z backendu
6. Wiadomość zaszyfrowana tym kluczem publicznym — odczytać może tylko odbiorca swoim prywatnym kluczem

Strumień real-time w `WebSocketService`:

Observable	Zdarzenie Socket.IO	Opis
<code>messageReceived\$</code>	<code>message_received</code>	Nowa wiadomość w konwersacji
<code>userTyping\$</code>	<code>user_typing</code>	Wskaźnik "pisze..."
<code>messageDelivered\$</code>	<code>message_delivered</code>	Potwierdzenie dostarczenia
<code>messageRead\$</code>	<code>message_read</code>	Potwierdzenie przeczytania
<code>userOnline\$</code>	<code>user_online</code>	Użytkownik dołączył do pokoju
<code>userOffline\$</code>	<code>user_offline</code>	Użytkownik opuścił pokój
<code>notification\$</code>	<code>notification</code>	Powiadomienie push z backendu

6.9 audit/ — Historia zmian (Audit Log)

Ścieżka URL	<code>/audit-logs</code>
Dostęp	Admin, rapla_editor, lecturer_rapla_editor
API	<code>AuditApiService</code> → <code>/audit-logs</code>

Funkcjonalności:

- Logi pogrupowane po **dacie** i **użytkowniku** — powiązane zmiany w jednej "partii"
- Oznaczanie nowości** — wpisy dodane od ostatniego odwiedzenia wyróżnione kolorowo
- Filtry** — po typie encji i akcji (CREATE/UPDATE/DELETE)
- Polskie etykiety** — mapa `entityMap` tłumaczy techniczne nazwy modeli na czytelne nazwy (np. `TeachingLoadAssignment` → "Przydział godzin")
- Po wejściu na stronę: `acknowledgeChanges()` informuje backend, że użytkownik widział zmiany

6.10 profile/ — Profil użytkownika

Ścieżka URL	<code>/profile</code>
Dostęp	Wszyscy zalogowani

Ustawienia konta: zmiana avatara, hasła, przeglądanie danych profilu (email, rola, wydział, tytuły).

7. Routing i ochrona tras

Mapa wszystkich tras

Ścieżka URL	Komponent	Guard	Dostęp
<code>/login</code>	LoginPage	Brak	Wszyscy
<code>/change-password</code>	ChangePasswordPage	Brak	Wszyscy
<code>/home</code>	HomePage	authGuard	Zalogowani
<code>/hermonogram</code>	HermonogramPage	authGuard	Admin, planer, rapla_editor
<code>/konflikty</code>	KonfliktyPage	authGuard	Admin, planer
<code>/sale</code>	SaleListPage	authGuard	Admin
<code>/sale/new</code>	SalePage	authGuard	Admin
<code>/sale/:id/edit</code>	SalePage	authGuard	Admin
<code>/sale/dictionaries</code>	SaleDictionariesPage	authGuard	Admin
<code>/subjects</code>	SubjectListPage	authGuard	Admin
<code>/subjects/new</code>	SubjectPage	authGuard	Admin
<code>/subjects/:id/edit</code>	SubjectPage	authGuard	Admin
<code>/subjects/dictionaries</code>	SubjectDictionariesPage	authGuard	Admin
<code>/teaching-loads</code>	TeachingLoadsPage	authGuard	Admin
<code>/admin/users/new</code>	UserCreatePage	authGuard	Admin
<code>/admin/unavailability-notes</code>	UnavailabilityNotesListAdminComponent	authGuard	Admin
<code>/chat</code>	ChatPage	authGuard	Zalogowani
<code>/audit-logs</code>	AuditLogsPage	authGuard	Admin, rapla_editor
<code>/profile</code>	ProfilePage	authGuard	Zalogowani

Ścieżka URL	Komponent	Guard	Dostęp
<code>/</code> (root)	—	redirect	→ <code>/login</code>
<code>/**</code> (inne)	—	redirect	→ <code>/login</code>

Lazy loading

Każda trasa ładuje swój komponent dopiero gdy użytkownik do niej nawiguje — przez `loadComponent: () => import('./...')`. Dzięki temu przeglądarka pobiera tylko kod niezbędny do wyświetlenia bieżącej strony, co przyspiesza czas inicjalnego ładowania aplikacji.

Role użytkowników

Rola	Dostęp do menu
<code>admin</code>	Wszystko: Dezyderaty, Harmonogram, Konflikty, Sale, Przedmioty, Przydziały godzin, Dodaj użytkownika, Notatki niedostępności, Historia zmian, Profil, Czat
<code>planner</code>	Dezyderaty, Harmonogram, Konflikty, Profil, Czat
<code>rapla_editor</code>	Dezyderaty, Harmonogram, Historia zmian, Profil, Czat
<code>lecturer</code>	Dezyderaty, Profil, Czat
<code>lecturer_rapla_editor</code>	Dezyderaty, Historia zmian, Profil, Czat

8. Bezpieczeństwo

Środki bezpieczeństwa zastosowane w projekcie

Zagrożenie	Zastosowane zabezpieczenie
XSS (Cross-Site Scripting) — wstrzyknięcie złośliwego JS	Angular automatycznie escapuje wszystkie wartości interpolowane w szablonach HTML. Refresh Token w HttpOnly Cookie — niedostępny dla JavaScript.
Kradzież tokenów	Access Token przechowywany tylko w pamięci RAM (nie localStorage). Refresh Token tylko w HttpOnly Cookie.
Unauthorized access — dostęp do chronionych stron	<code>authGuard</code> blokuje rendering chronionego widoku. Backend weryfikuje token na każdym endpointzie.
Przechwycenie wiadomości czatu	Szyfrowanie end-to-end RSA-OAEP 2048-bit — nawet administrator serwera nie może odczytać treści wiadomości bez klucza prywatnego odbiorcy.
Słabe hasła	<code>passwordStrengthValidator</code> wymaga min. 8 znaków, wielkich liter, cyfr, znaków specjalnych. Hasło nie może zawierać imienia/nazwiska.
Wygaśnięcie sesji	Access Token z krótkim TTL + automatyczny refresh. Bufor 10s zapobiega wyścigom.
Race condition w refresh	Singleton <code>refreshInFlight\$</code> z <code>shareReplay(1)</code> — wiele równoległych 401 wywołuje tylko jeden request do <code>/auth/refresh</code> .

9. Wdrożenie (Docker + nginx)

Dockerfile — wieloetapowy build

Proces budowania aplikacji skonteneryzowany w Dockerze — dwa etapy:

Etap 1 — Build (Node.js)

Instalacja zależności npm, uruchomienie `ng build --configuration production`.
Wynikiem są zoptymalizowane pliki statyczne w katalogu `www/`.

Etap 2 — Serve (nginx)

Skopiowanie plików z etapu 1 do obrazu nginx. Skonfigurowano `nginx.conf` specjalnie dla SPA — każda ścieżka zwraca `index.html`, Angular Router obsługuje resztę.

Polecenia

```
# Tryb deweloperski
npm run dev                # ng serve --hmr (Hot Module Replacement)

# Build produkcyjny
npm run build              # ng build

# Docker
docker build -t zpi-desktop .
docker run -p 80:80 zpi-desktop

# Testy
npm test                  # Jasmine + Karma
```